

C++ Módulo 01

O GUIA DEFINITIVO E BIBLIOGRÁFICO

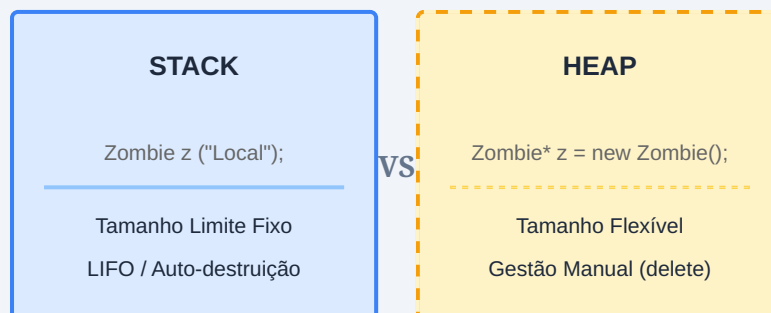
*Elaborado por: **Leonardo La Rocque**
42 Porto - Engenharia de Software*

I. Gestão Dinâmica de Memória (ex00 & ex01)

1.1. O Paradigma Stack vs. Heap

A arquitetura de memória do C++ divide-se em duas regiões críticas operadas de formas fundamentalmente distintas. A **Stack** (pilha de execução) é rigorosamente gerida pelo compilador através do princípio LIFO (Last-In, First-Out). A alocação é estática, o tempo de acesso é extremamente rápido e a destruição do objeto é garantida assim que o *scope* termina.

Em oposição, a **Heap** (amontoado) fornece um espaço vasto para alocação de memória dinâmica. O controlo passa inteiramente para as mãos do programador através da palavra-chave **new**. A grande contrapartida desta liberdade é o risco do *Memory Leak*: a memória permanece alocada indefinidamente até que o programador exija a sua libertação explícita com a palavra-chave **delete**.



```
// Exemplo Bibliográfico: Ciclo de Vida
void runSimulation() {
    // Alocação na Stack (rápida, morre no final da função)
    Zombie stackZombie("StackFoo");

    // Alocação na Heap (sobrevive à função)
    Zombie* heapZombie = new Zombie("HeapBar");

    // Responsabilidade explícita do Engenheiro:
    delete heapZombie;
} // stackZombie é destruído automaticamente aqui.
```

1.2. A Rasteira dos Arrays Dinâmicos

A instanciação de múltiplos objetos contíguos (**new**[]) exige invariavelmente o operador de vetorização **delete**[]. Quando alocamos `new Zombie[N]`, o compilador injeta secretamente os bytes necessários antes do bloco de memória para memorizar o valor N. Usar o `delete` padrão apenas acionará o destrutor do primeiro objeto, resultando num *Undefined Behavior* (Comportamento Indefinido).

II. Referências e Composição de Classes (ex02 & ex03)

2.1. Anatomia de uma Referência (&)

A introdução das **Referências** é um dos grandes marcos do C++ em relação ao C. Enquanto um ponteiro é uma variável física independente que guarda um endereço, a referência atua como um apelido (alias) sintático de uma variável pré-existente.

O que acontece por baixo do capô?

Ao nível do assembly, as referências são frequentemente implementadas como *Constant Pointers*, ou seja, `Type* const ptr`. Contudo, o compilador abstrai o uso do asterisco `*`, permitindo-nos aceder aos membros com a notação de ponto `.`, garantindo segurança estática.

```
std::string brain = "HI THIS IS BRAIN";

// Pontoeiro: Variável mutável que aponta para moradas
std::string* stringPTR = &brain;

// Referência: Apelido imutável (tem de ser inicializado na criação)
std::string& stringREF = brain;
```

2.2. Tomada de Decisão: Pointer vs Reference

O Design de Software (OOP) exige decisões arquiteturais precisas. A diferença entre o HumanA e o HumanB não é apenas sintática, é semântica:

- **Composição Estrita (Referência):** Se uma entidade não faz sentido físico ou lógico sem um determinado atributo (ex: HumanA *nasce e morre* com a sua Weapon), exigimos uma `Weapon&` no construtor.
- **Agregação Flexível (Ponteiro):** Se a posse de um atributo é opcional ou mutável no tempo (ex: HumanB pode estar desarmado e apanhar uma arma depois), utilizamos um ponteiro `Weapon*` inicializado a `NULL`, com a devida verificação de segurança (`if (ptr != NULL)`) antes da utilização.

III. File Streams e Strings (ex04)

3.1. A Abstração de I/O em C++

A biblioteca `<fstream>` eleva a manipulação de ficheiros de `FILE*` (C) para o paradigma Orientado a Objetos. Criamos canais diretos (Streams) entre o disco rígido e o programa. A gestão de *File Descriptors* torna-se mais segura, embora o encerramento explícito (`.close()`) se mantenha como a *best-practice* principal contra o esgotamento de recursos do Sistema Operativo.

3.2. A Engenharia do *Find and Replace*

O desafio de substituir texto sem recorrer à função nativa da `std::string` requer uma sinergia rigorosa entre os métodos `find()`, `erase()` e `insert()`. O erro clássico deste algoritmo é o Loop Infinito provocado por uma substituição de *string* que contém a si mesma.

O Risco do Loop Infinito: Se substituirmos "A" por "AA", a reinicialização da procura no mesmo índice voltará a ler o "A" recém-inserido indefinidamente.

```
std::size_t pos = 0;
// npos atua como um código de falha absoluto (o maior size_t possível)
while ((pos = line.find(s1, pos)) != std::string::npos)
{
    line.erase(pos, s1.length());
    line.insert(pos, s2);

    // Deslocamento estratégico do cursor de memória para evitar loops:
    pos += s2.length();
}
```

IV. O Submundo do C++: Ponteiros e Switch (ex05 & ex06)

4.1. Ponteiros para Funções Membro

Evitar árvores de decisão gigantescas (*if/else if* em cascata) é um princípio fundamental de *Clean Code*. O C++ fornece a capacidade de construir tabelas de despacho (Dispatch Tables) utilizando arrays de **Pointers to Member Functions**.

A sua sintaxe é notoriamente complexa porque uma função membro não é apenas um endereço de código; ela possui um parâmetro oculto indispensável: o ponteiro `this`. Para invocar o ponteiro, o compilador necessita que associemos o endereço armazenado a um objeto real.

```
// Declaração de um array de ponteiros para funções membro void(void)
void (Harl::*methods[])(void) = {
    &Harl::debug, &Harl::info, &Harl::warning, &Harl::error
};

// Invocação correta acoplado o ponteiro this à tabela
(this->*methods[i])();
```

4.2. Otimização com o Switch e o Efeito Fall-Through

A instrução `switch` em C++ opera como uma *Jump Table* nativa ao nível do processador, sendo infinitamente mais eficiente do que validações de strings iterativas. Ao traduzir previamente uma string para um identificador Inteiro (`int`), desbloqueamos esta funcionalidade.

A ausência da instrução `break`; desencadeia o **Fall-Through**: a execução desmorona proposadamente em cascata para os blocos case sucessivos. Compiladores modernos, devido à sua rigorosa arquitetura de segurança (ex: `-Werror=implicit-fallthrough`), penalizam esta omissão a menos que o programador declare de forma categórica a sua intenção arquitetural através do comentário de sistema `/* fall through */`.

[Fim do Documento Bibliográfico]